



Assiut University
Faculty of Computers & Information
Computer Science Department



Intelligent Systems

DR.SARA SALAH MOHAMED

Grading

Assessment	Grade %
Final Exam	50%
Class Work	6%
Mid-Term Exam	15%
Project	24%

Text Book

INTELLIGENT SYSTEMS REFERENCE LIBRARY
Volume 17

Crina Grosan
Ajith Abraham

Intelligent Systems

A Modern Approach

 Springer

Content

◀ Evolutionary Algorithms

Introduction

▶ How to Build an Evolutionary Algorithm?

◀ Genetic Algorithms

Representing the
Individuals

Selection Mechanisms

Selection Mechanisms

Variation Operators

Population Models

Survivor Selection and
Reinsertion

The Basic Genetic
Algorithm

Evolutionary Algorithm

Step 1. Set $t = 0$;

Step 2. Randomly initialize the population $P(t)$.

Repeat

 Step 3.1. Evaluate individuals from $P(t)$;

 Step 3.2. Selection on $P(t)$

 Let $P'(t)$ be the set of selected individuals.

 Step 3.3. Crossover on $P'(t)$;

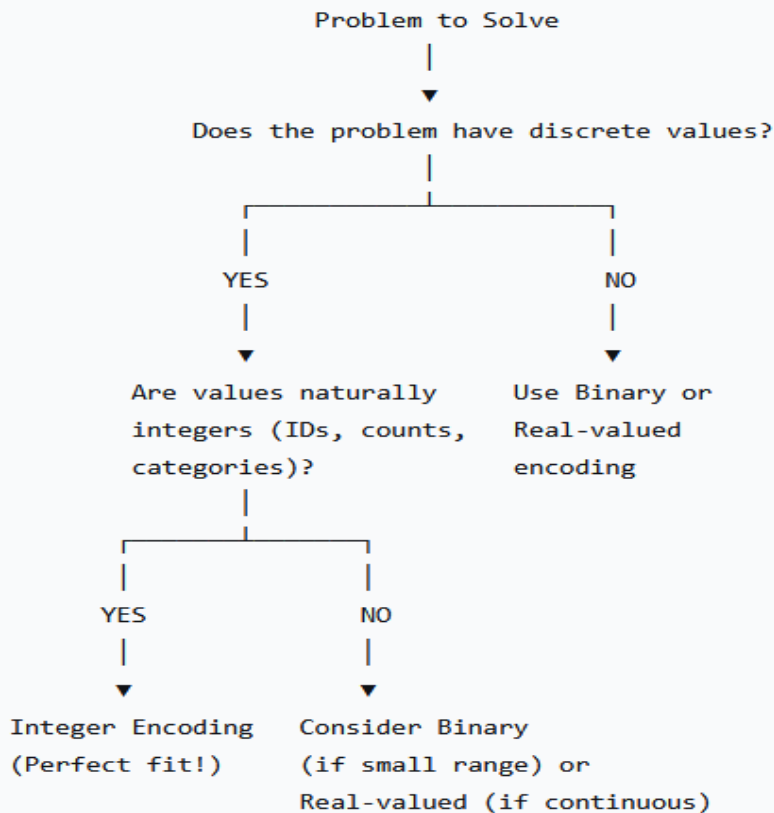
 Step 3.4. Mutation on $P'(t)$.

 Step 3.5. Survival on $P'(t)$.

 Step 3.6. $t = t + 1$.

Until $t = \text{number of iterations}$

When to Use Integer Encoding?



When NOT to Use Integer Encoding?

Case 1: Pure binary decisions

```
if problem_type == "yes/no decisions":  
    use_binary_encoding() # [1,0,1,1,0] is more natural
```

Case 2: Continuous optimization

```
elif problem_type == "continuous parameters":  
    use_real_encoding() # [3.14, 2.718, 1.618] better precision
```

Case 3: Small discrete sets (< 8 values)

```
elif n_values <= 8:  
    consider_binary_encoding() # More theoretical foundation
```

2-Integer Representation

- Integer representation uses whole numbers (integers) as genes in a chromosome. Each position can take values from a specified set or range of integers, making it ideal for problems with discrete, countable options.

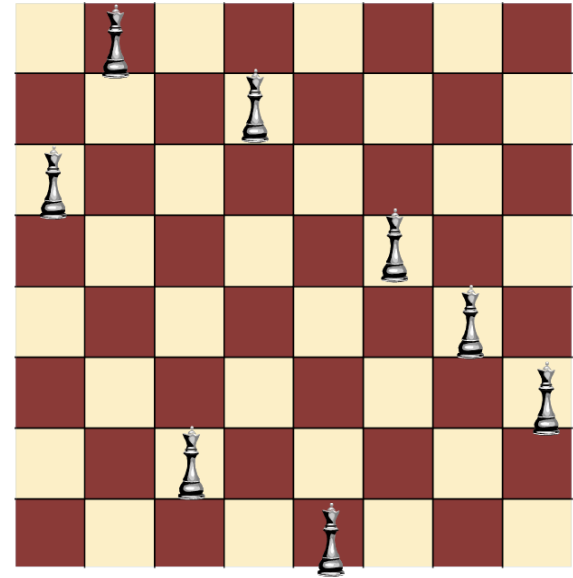
Why Use Integer Representation?

1. **Natural mapping** - Many problems inherently use integers (colors, quantities, IDs)
2. **No decoding needed** - Direct interpretation saves computation
3. **Avoids binary complexity** - No bit counting or precision issues
4. **Maintains constraints** - Easy to enforce value ranges
5. **Better for combinatorial problems** - Permutations, assignments, scheduling

Chromosome = [3, 7, 1, 4, 2, 8, 5, 6]

↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑

Genes (each is an integer value)



Graphical representation of the solution (2 4 1 6 7 8 3 5) for the 8-queens

Types of Integer Representation

1. Value Encoding (Simple Integers)

Each gene can be any integer within a range

Problem: Assign quantities to 5 products (0-100 units each)

Chromosome: [45, 67, 12, 89, 33]

Meaning: Product1=45 units, Product2=67 units, etc.

2. Permutation Encoding (Order-Based)

Each integer appears exactly once, represents a sequence or order.

Problem: Traveling Salesman Problem (visit 8 cities)

Chromosome: [3, 1, 4, 2, 7, 5, 8, 6]

Meaning: Visit city3 → city1 → city4 → city2 → city7 → city5 → city8 → city6

3. Multi-Parameter Encoding

Different genes represent different types of parameters

Problem: Design a car (color, engine, wheels, transmission)

Domains: Color(1-6), Engine(1-4), Wheels(1-3), Transmission(1-2)

Chromosome: [3, 2, 1, 2]

Meaning: Blue car, V6 engine, Alloy wheels, Automatic

Crossover or Recombination for Integer Representation

❑ Crossover for Value Encoding

1-Single-Point Crossover: cut both parents at the same point and swap tails as we learn in binary representation .

parent1: [45, 67, 12 | 89, 33, 21, 54, 78]

Parent2: [23, 56, 91 | 34, 62, 85, 47, 19]

↑

Cut point

Child1: [45, 67, 12 | 34, 62, 85, 47, 19]

Child2: [23, 56, 91 | 89, 33, 21, 54, 78]

2 Two-Point Crossover

Selects a segment between two points and swaps it between parents.

Parent1: [45, 67 | 12, 89, 33 | 21, 54, 78]

Parent2: [23, 56 | 91, 34, 62 | 85, 47, 19]

↑

Point1

↑

Point2

Child1: [45, 67 | 91, 34, 62 | 21, 54, 78]

Child2: [23, 56 | 12, 89, 33 | 85, 47, 19]

3. Uniform Crossover

Each gene is independently chosen from either parent with equal probability.

Parent1: [45, 67, 12, 89, 33, 21, 54, 78]

Parent2: [23, 56, 91, 34, 62, 85, 47, 19]

Mask: [1, 0, 1, 0, 1, 0, 1, 0]

Child1: [45, 56, 12, 34, 33, 85, 54, 19] (from P1 where mask=1, else P2)

Child2: [23, 67, 91, 89, 62, 21, 47, 78] (opposite of Child1)

4. Arithmetic Crossover

Creates children as weighted averages of parents. Three types (simple, single arithmetic and whole arithmetic)

➤ whole arithmetic

p1 = [10, 20, 30, 40, 50]

p2 = [90, 80, 70, 60, 50]

Alpha = 0.3 (Child1 70% P2, 30% P1)

Position 0: Child1 = $0.3 \times 10 + 0.7 \times 90 = 3 + 63 = 66$

Child2 = $0.7 \times 10 + 0.3 \times 90 = 7 + 27 = 34$

Position 1: Child1 = $0.3 \times 20 + 0.7 \times 80 = 6 + 56 = 62$

Child2 = $0.7 \times 20 + 0.3 \times 80 = 14 + 24 = 38$

... and so on

❑ Crossover for Permutation Encoding

In permutation encoding, each integer appears exactly once. Standard crossover would create invalid solutions with duplicates.

1. Partially mapped crossover Partially mapped crossover (PMX): has been proposed in [19] as a recombination operator for the TSP. There exist now several variants of it.

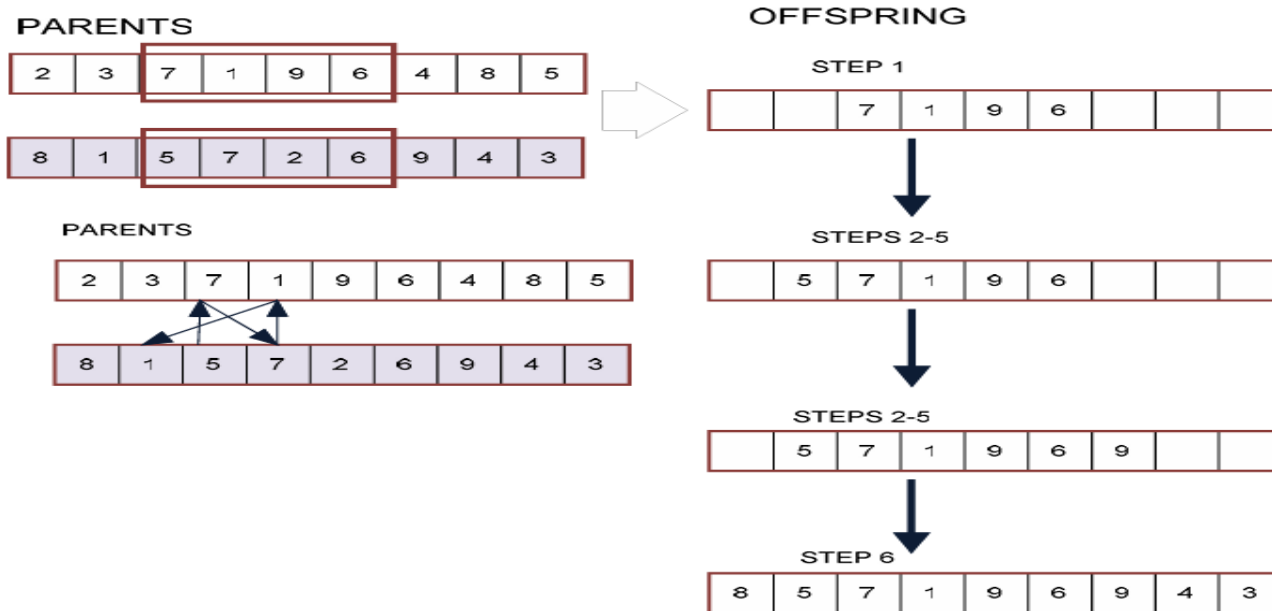


Fig. 14.21 PMX operator.

Example PMX:

Step 1: Select segment

Parent1: [3, 1, | 4, 2, 7 |, 5, 8, 6]

Parent2: [2, 4, | 1, 3, 6 |, 8, 5, 7]

↑ ↑
point1 point2

Step 2: Copy segment from Parent1 to child

Child: [-, -, | 4, 2, 7 |, -, -, -]

Step 3: Create mapping from segment

Compare positions in the segment:

Position 2: P1=4, P2=1 → 4 ↔ 1

Position 3: P1=2, P2=3 → 2 ↔ 3

Position 4: P1=7, P2=6 → 7 ↔ 6

Mapping: {4:1, 2:3, 7:6}

Step 4: Fill remaining positions from Parent2

Position 0: P2[0]=2 → 2 maps to 3 → Child[0]=3

Position 1: P2[1]=4 → 4 maps to 1 → Child[1]=1

Position 5: P2[5]=8 → 8 not in map → Child[5]=8

Position 6: P2[6]=5 → 5 not in map → Child[6]=5

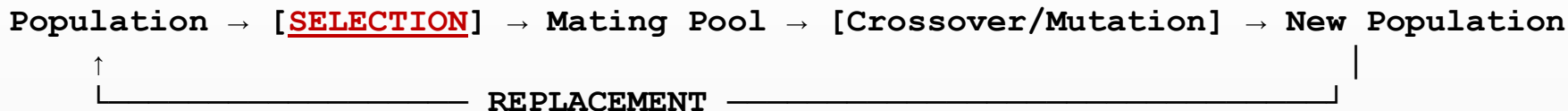
Position 7: P2[7]=7 → 7 maps to 6 → Child[7]=6

Final Child: [3, 1, 4, 2, 7, 8, 5, 6]

Selection

Selection is the process of choosing individuals from the current population to serve as parents for the next generation. It's the driving force that pushes the population toward better solutions.

The Role of Selection

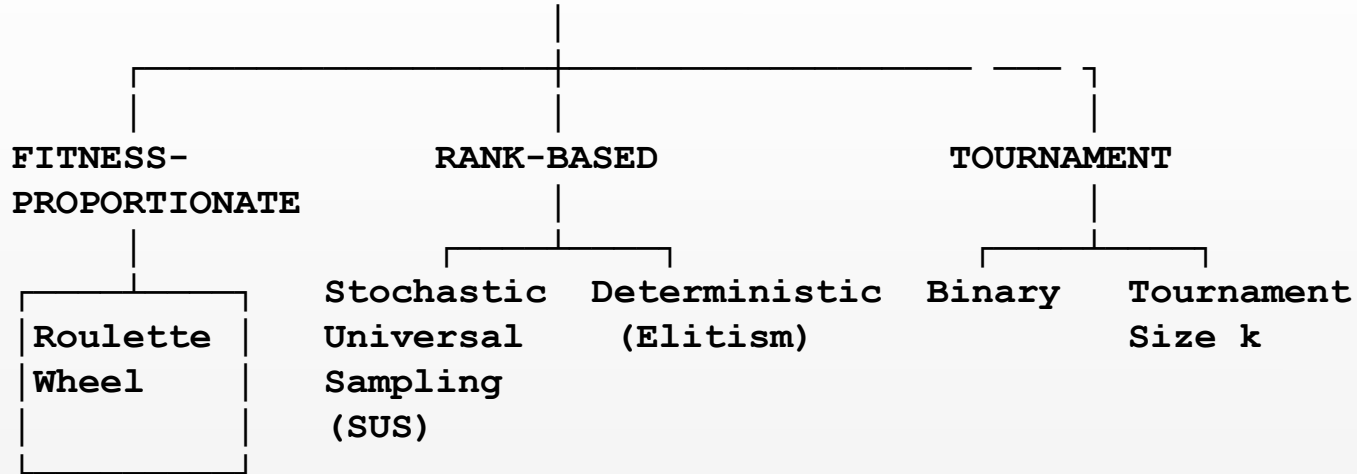


Key Properties of Selection Methods

1. **Selection Pressure:** How strongly better individuals are favored
2. **Population Diversity:** How well less-fit individuals are preserved
3. **Convergence Speed:** How quickly the population homogenizes
4. **Computational Complexity:** Time required to perform selection

Classification of Selection Methods

SELECTION METHODS



1 Roulette Wheel Selection

Also known as **Fitness-Proportionate Selection**. Individuals are selected with probability proportional to their fitness.

This selection mechanism was introduced in [9] (see also [10]). Let us denote by f_i the fitness of the i -th individual in the population and by N the population size (number of individuals in the population). Thus, the probability p_i that individual i is selected for mating (for recombination) is given by:

$$p_i = \frac{f_i}{\sum_{i=1}^N f_i}$$

This means that the selection probability depends on the absolute fitness value of the individual compared to the absolute fitness values of the rest of the population.

There are some problems with this selection mechanism [2]:

- individuals that are much better than the rest take over the entire population very quickly and this leads to *premature convergence*;
- when fitness values are very close there is almost no selection pressure;

Number of individual	1	2	3	4	5	6	7	8	9
Fitness value	5.0	4.5	4.0	3.5	3.0	2.5	2.0	1.5	1.0
Selection probability	0.19	0.17	0.15	0.13	0.11	0.09	0.07	0.05	0.04

Fig. 14.8 Example of individuals, their fitness values and the corresponding selection probability.

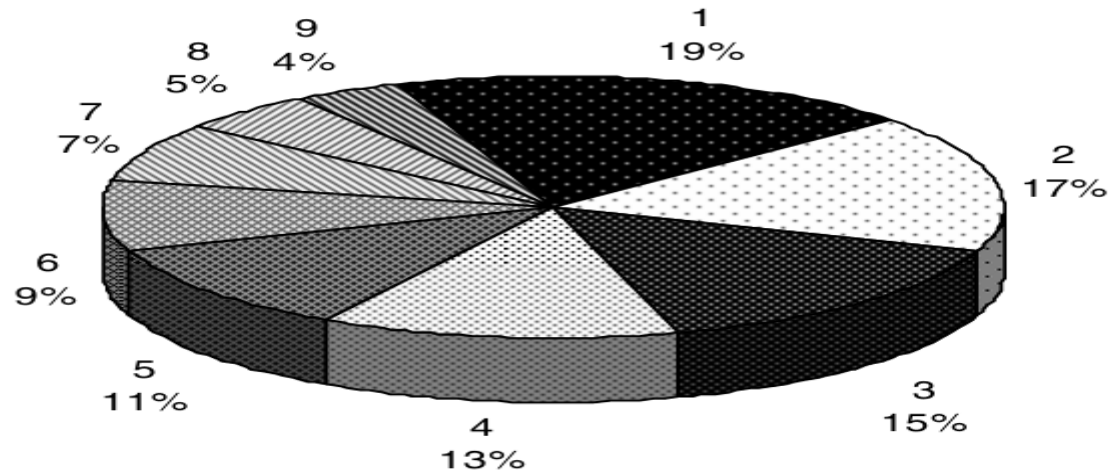
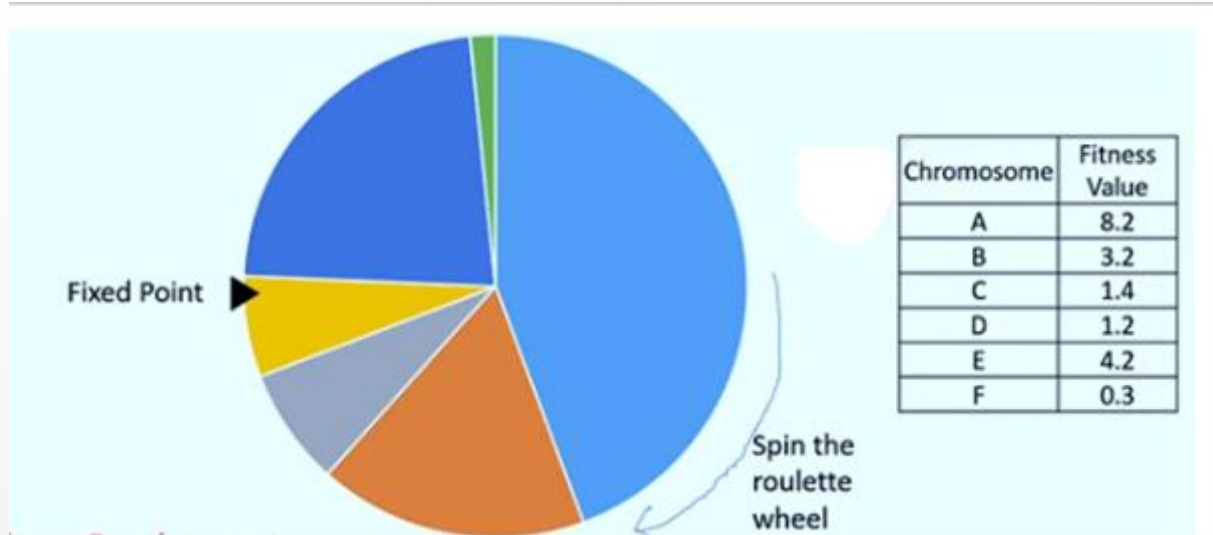


Fig. 14.10 Roulette wheel corresponding to the example.



جزاكم الله خيرا
وصل اللهم على سيدنا محمد وعلى آله وصحبه وسلم